

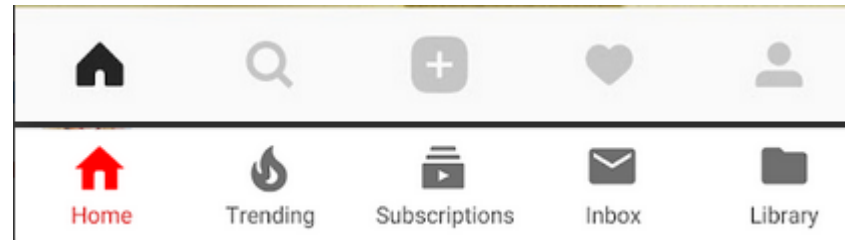
Tema 3.6.2: Menú de navegación – Botton Navigation

Android 

 Kotlin

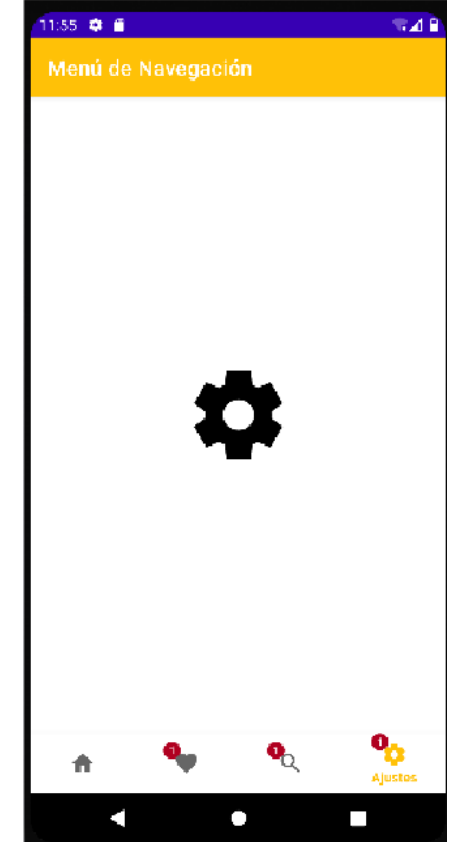
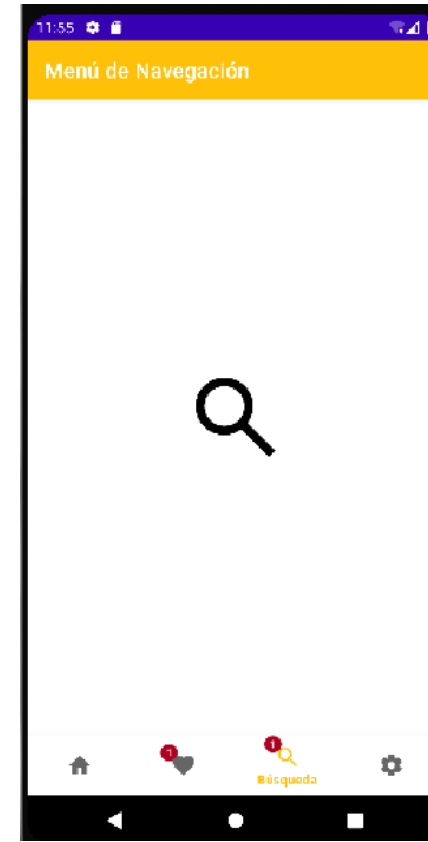
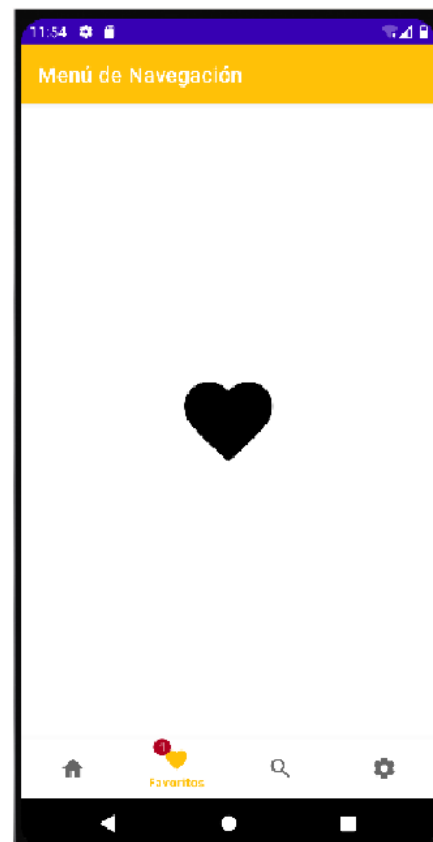
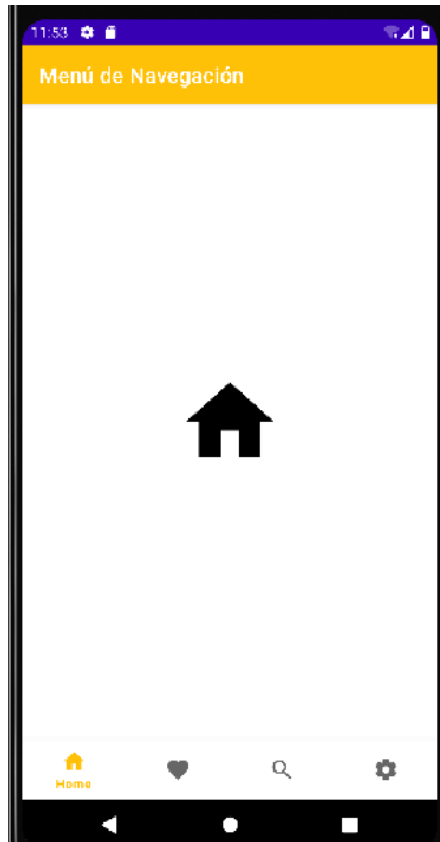
Menú de navegación

La **barra o menú de navegación inferior (Bottom Navigation)** es uno de los elementos de navegación esenciales definidos por Material Design. La ves a diario en aplicaciones tan populares como Spotify, Instagram, TikTok o YouTube. Muestra varios iconos, a veces acompañados por un texto, que dan acceso a las principales secciones de una aplicación.



Google aconseja que una barra de navegación inferior contenga como **máximo cinco elementos**. Si necesitas más, lo recomendado es un menú lateral de tipo Navigation drawer.

Menú de navegación



Añadiendo toolbars

Tiene una `Toolbar` arriba y un `BottomNavigationView` abajo. Ambos componentes se encuentran dentro de un `ConstraintLayout`.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/toolbar"
        android:layout_width="match_parent"
        android:layout_height="?attr/actionBarSize"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/bottom_navigation"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:menu="@menu/bottom_nav_menu"/>

</androidx.constraintlayout.widget.ConstraintLayout>
```



Creamos un recurso de menú

En primer lugar, crea el archivo XML (`bottom_nav_menu`) con la definición de la estructura del menú que deseas mostrar.

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">
  <item
    android:id="@+id/page_home"
    android:icon="@drawable/ic_home"
    android:title="@string/bottom_nav_home" />
  <item
    android:id="@+id/page_fav"
    android:icon="@drawable/ic_fav"
    android:title="@string/bottom_nav_fav" />
  <item
    android:id="@+id/page_search"
    android:icon="@drawable/ic_search"
    android:title="@string/bottom_nav_search" />
  <item
    android:id="@+id/page_settings"
    android:icon="@drawable/ic_settings"
    android:title="@string/bottom_nav_settings" />
</menu>
```

Creamos un recurso de menú

Ahora, descarga los iconos del Moodle y déjalos en la carpeta “res\drawable”

Incluye los siguientes elementos en el archivo “strings.xml”:

```
<resources>
  <string name="app_name">Menú de Navegación</string>
  <string name="bottom_nav_home">Home</string>
  <string name="bottom_nav_fav">Favoritos</string>
  <string name="bottom_nav_search">Búsqueda</string>
  <string name="bottom_nav_settings">Ajustes</string>
  <!-- TODO: Remove or change this placeholder text -->
  <string name="hello_blank_fragment">Hello blank fragment</string>
</resources>
```

Creamos un recurso de menú

Incluye los siguientes elementos en el archivo “colors.xml”:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <color name="black">#FF000000</color>
  <color name="white">#FFFFFFFF</color>
  <color name="primary">#FFC107</color>
  <color name="primaryDark">#000000</color>
</resources>
```

Incluye los siguientes elementos en el archivo “themes.xml” para personalizarla. [Documentación oficial](#) :

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <style name="Theme.Navigation_menu" parent="Theme.MaterialComponents.DayNight.NoActionBar">
    <item name="colorPrimary">@color/primary</item>
    <item name="toolbarStyle">@style/Widget.MaterialComponents.Toolbar.PrimarySurface</item>
  </style>
</resources>
```

Creamos un recurso de menú

En nuestra clase principal `MainActivity.kt`, tendremos que localizar la toolbar con `findViewById` y activarla.

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    setContentView(R.layout.activity_main)  
  
    val toolbar: Toolbar = findViewById(R.id.toolbar)  
    setSupportActionBar(toolbar)  
}
```

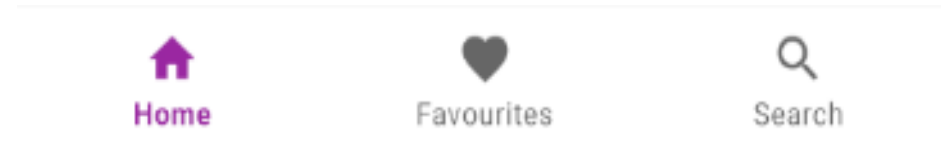
Se hace referencia al **objeto Toolbar** mediante su identificador **R.id.toolbar**, que está definido en el archivo XML dentro de la carpeta `res/layout`. También se está configurando esta **Toolbar** como la **barra de herramientas de la actividad** utilizando el método `setSupportActionBar(toolbar)`

Probamos el menú de navegación

Los ítems solo muestran el icono, y la etiqueta queda reservada para el ítem seleccionado. Esto ocurre cuando el menú posee **cuatro o más ítems**.



Con menos, **icono y etiqueta** se muestran siempre:

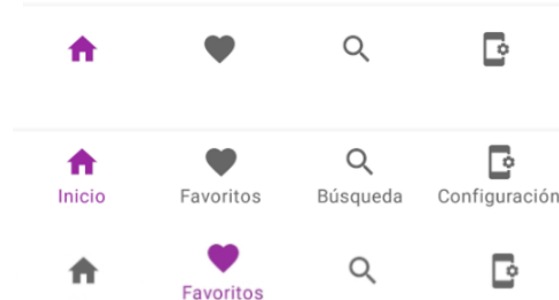


Cambiamos este comportamiento con la propiedad `labelVisibilityMode`:

```
app:labelVisibilityMode="unlabeled"
```

```
app:labelVisibilityMode="labeled"
```

```
app:labelVisibilityMode="selected"
```



Pantallas de ejemplo con fragments

Vamos a realizar un ejemplo en el que cada ítem muestra un `fragment` o fragmento en el centro de la pantalla que a su vez **muestre el icono** correspondiente al ítem pulsado.

Un **fragmento es un componente reutilizable y combinable** que encapsula una interfaz gráfica y «vive» dentro de las actividades. En la práctica, consiste en **una clase que gestiona un layout** de acuerdo a un ciclo de vida bien definido.

En primer lugar, crea el `layout` del fragmento: (`fragment_page`)

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <ImageView
        android:id="@+id/imageView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:scaleX="1"
        android:scaleY="1" />

</FrameLayout>
```

La clase del fragmento

Creamos la **lógica (kotlin) de un fragmento** (lo metemos dentro de una carpeta (**Package**) llamada “fragmentos”). Y llamamos al archivo kotlin que estamos creando “PageFragment” Modificamos el código con y dejamos solo este:

```
class PageFragment : Fragment() {
    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?): View? {
        val layout = inflater.inflate(R.layout.fragment_page, container, false)
        val icon = arguments?.getInt(ARG_ICON, 0) ?: 0
        layout.findViewById<View>(R.id.imageView).setBackgroundResource(icon)
        return layout
    }
    companion object {
        private const val ARG_ICON = "ARG_ICON"

        fun newInstance(@DrawableRes iconId: Int): PageFragment {
            val fragment = PageFragment()

            val args = Bundle()
            args.putInt(ARG_ICON, iconId)
            fragment.arguments = args

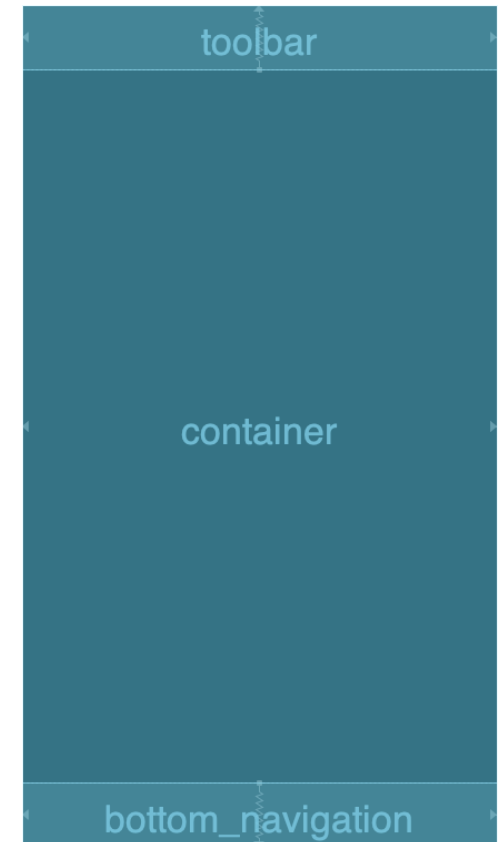
            return fragment
        }
    }
}
```

Incluyendo el fragmento en pantalla

Ahora bien, ¿cómo incluyes el fragmento en la pantalla? Necesitamos un contenedor en el que incrustarlo desde el código en “activity_main.xml”.

Podemos utilizar el `FragmentManager` de la paleta de vistas.

```
<androidx.fragment.app.FragmentContainerView
    android:id="@+id/container"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:layout_constraintBottom_toBottomOf="@+id/bottom_navigation"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="@+id/toolbar" />
```



Añadimos los listeners

Capturaremos y procesaremos la pulsación de los elementos del menú con un listener de tipo `OnNavigationItemSelectedListener` vinculado al `BottomNavigationView`:

```
private fun setupBottomMenu() {  
    bottomNavigationView = findViewById(R.id.bottom_navigation)  
    bottomNavigationView.setOnNavigationItemSelectedListener { item -> onItemSelectedListener(item) }  
    bottomNavigationView.setSelectedItemId(R.id.page_home)  
}  
  
private fun onItemSelectedListener(item: MenuItem): Boolean {  
    when (item.itemId) {  
        R.id.page_home -> {  
            showPageFragment(R.drawable.ic_home, R.string.bottom_nav_home)  
            return true  
        }  
        R.id.page_fav -> {  
            showPageFragment(R.drawable.ic_fav, R.string.bottom_nav_fav)  
            return true  
        }  
        R.id.page_search -> {  
            showPageFragment(R.drawable.ic_search, R.string.bottom_nav_search)  
            return true  
        }  
        R.id.page_settings -> {  
            showPageFragment(R.drawable.ic_settings, R.string.bottom_nav_settings)  
            return true  
        }  
        else -> throw IllegalArgumentException("Item not implemented: ${item.itemId}")  
    }  
}  
  
private fun showPageFragment(@DrawableRes iconId: Int, @StringRes title: Int) {  
    val fragment = PageFragment.newInstance(iconId)  
    supportFragmentManager.beginTransaction()  
        .setCustomAnimations(R.anim.bottom_nav_enter, R.anim.bottom_nav_exit)  
        .replace(R.id.container, fragment)  
        .commit()  
}
```

Animaciones de los fragments

Para la creación de animaciones simples, podemos crear un Resource de tipo `anim`. Igual que con la creación de menús o layouts, **res>New>Android Resource File**. Le pondremos el nombre y en Resource Type seleccionaremos Animation. A continuación hay dos ejemplos que utilizaremos de entrada y de salida para los fragments.

```
<?xml version="1.0" encoding="utf-8"?>
<set
xmlns:android="http://schemas.android.com/apk/res/android">
  <scale
    android:duration="1000"
    android:fromXScale="0"
    android:fromYScale="0"
    android:pivotX="50%"
    android:pivotY="50%"
    android:repeatMode="restart"
    android:toXScale="1.0"
    android:toYScale="1.0" />

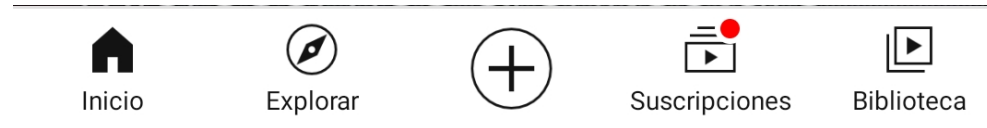
  <alpha
    android:duration="1000"
    android:fromAlpha="0.0"
    android:toAlpha="1" />
</set>
```

```
<?xml version="1.0" encoding="utf-8"?>
<set
xmlns:android="http://schemas.android.com/apk/res/android">
  <scale
    android:duration="1000"
    android:fromXScale="1.0"
    android:fromYScale="1.0"
    android:pivotX="50%"
    android:pivotY="50%"
    android:repeatMode="restart"
    android:toXScale="0"
    android:toYScale="0" />

  <alpha
    android:duration="1000"
    android:fromAlpha="1.0"
    android:toAlpha="0" />
</set>
```

Indicadores con BadgeDrawable

Las iconos del menú pueden incluir un indicador superpuesto en una esquina, denominado **badge** o **insignia**, con el fin llamar la atención del usuario. El caso más típico es avisar que en la sección de notificaciones hay mensajes pendientes de leer.



Material Components provee un **mecanismo de insignias** integrado a la perfección con `BottomNavigationView`. Para usarlo, llama al método `BottomNavigationView.getOrCreateBadge` a fin de obtener un objeto `BadgeDrawable` asociado al ítem del menú que quieras. Ese objeto representa a la insignia del ítem.

```
val badge = bottomNavigationView.getOrCreateBadge(R.id.page_fav)
```

Modificando las badges/insignias

Una vez tengas el objeto `BadgeDrawable`, controlamos la visibilidad de la insignia con el método `setVisible`. De manera predeterminada, la insignia permanece oculta. Además, si lo deseas, con `setNumber` añades un número a la insignia. Así, por ejemplo, puedes informar al usuario cuántas notificaciones tiene pendientes de lectura.

Aquí tienes una demostración. Observa que `setNumber` ya muestra la insignia, haciendo redundante la llamada a `setVisible`:

```
val badge = bottomNavigationView.getOrCreateBadge(R.id.page_fav)
bottomNavigationView.getOrCreateBadge(R.id.page_fav).number = 1000
bottomNavigationView.getOrCreateBadge(R.id.page_settings).isVisible = true
```


Modificando las badges/insignias

Algunas consideraciones acerca de las badges:

- Su color es el `colorError` del tema.
- Personalizas el número máximo de caracteres del número con el método `setMaxCharacterCount`.
- Cambiaremos su posición con el método `setBadgeGravity`. En la siguiente imagen tienes las cuatro opciones posibles, definidas como constantes en `BadgeDrawable`:



```
badge.badgeGravity = BadgeDrawable.TOP_START
```

Usando las badges/insignias

Primero definiremos como variable de la clase el item de menu.

```
private lateinit var bottomNavigationView: BottomNavigationView
```

Dentro de la función que carga el BottomMenu `setupBottomMenu`, vamos a iniciar todas las insignias, pero las mantendremos ocultas hasta que hagamos el primer click.

```
private fun setupBottomMenu() {  
    bottomNavigationView = findViewById(R.id.bottom_navigation)  
    bottomNavigationView.setOnItemSelectedListener { item ->  
onItemSelectedListener(item) }  
    bottomNavigationView.setSelectedItemId(R.id.page_home)  
  
    bottomNavigationView.menu.forEach { menuItem ->  
        val badge = bottomNavigationView.getOrCreateBadge(menuItem.itemId)  
        badge.isVisible = false  
        badge.badgeGravity = BadgeDrawable.TOP_START  
    }  
}
```

Usando las badges/insignias

Primero definiremos como variable de la clase las listas que contendrán si hemos hecho click y cuántas veces.

```
private val badgeCounts = mutableMapOf<Int, Int>()
private val badgeClicks = mutableSetOf<Int>()
```

En la función que carga el BottomMenu `setupBottomMenu`, vamos a inicializar todas las insignias a 0.

```
private fun setupBottomMenu() {
    bottomNavigationView = findViewById(R.id.bottom_navigation)
    bottomNavigationView.setOnItemSelectedListener { item ->
onItemSelectedListener(item) }
    bottomNavigationView.setSelectedItemId(R.id.page_home)

    bottomNavigationView.menu.forEach { menuItem ->
        val badge = bottomNavigationView.getOrCreateBadge(menuItem.itemId)
        badge.isVisible = false
        badge.badgeGravity = BadgeDrawable.TOP_START
        badgeCounts[menuItem.itemId] = 0
    }
}
```

Usando las badges/insignias

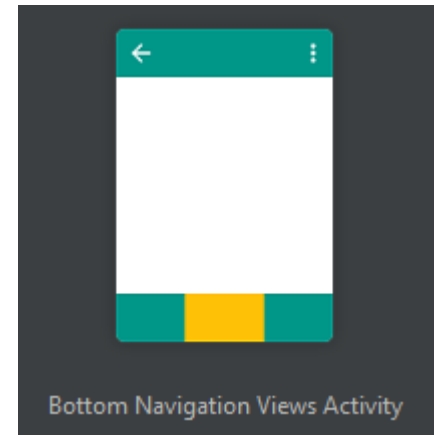
Dentro de la función que se encarga de los clicks en los ítems del menú `onItemSelectedListener`, vamos a añadir la lógica que contará los clicks en cada ítem y actualizará las badges.

```
private fun onItemSelectedListener(item: MenuItem): Boolean {  
  
    val itemId = item.itemId  
  
    if (!badgeClicks.contains(itemId)) {  
        val badge = bottomNavigationView.getOrCreateBadge(itemId)  
        badge.isVisible = true  
        badgeClicks.add(itemId)  
    }  
    val currentCount = badgeCounts[itemId] ?: 0  
    badgeCounts[itemId] = currentCount + 1  
  
    val badge = bottomNavigationView.getOrCreateBadge(itemId)  
    badge.number = badgeCounts[itemId] ?: 0  
    when (item.itemId) {
```

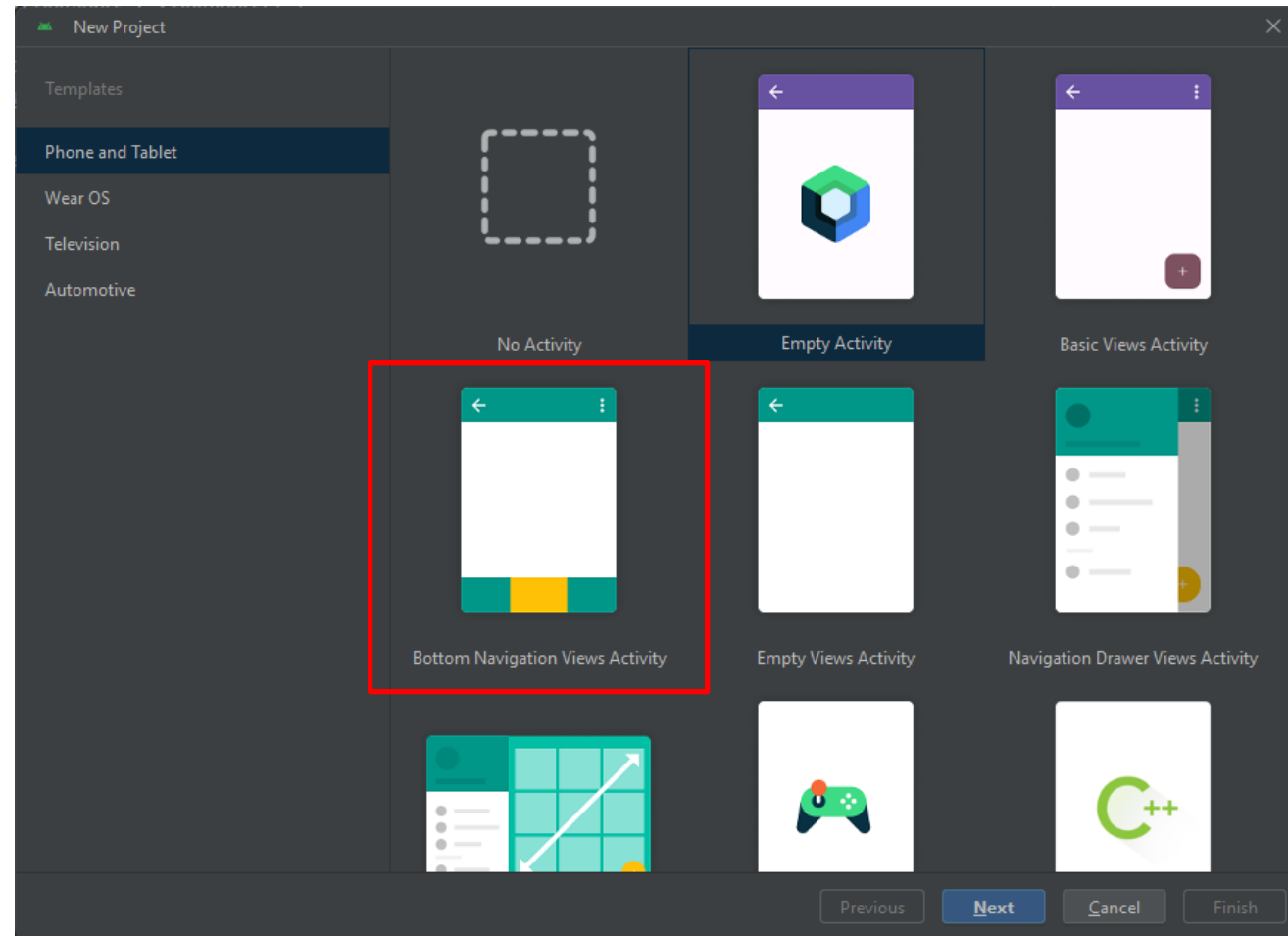


Menú de navegación Bottom Navigation

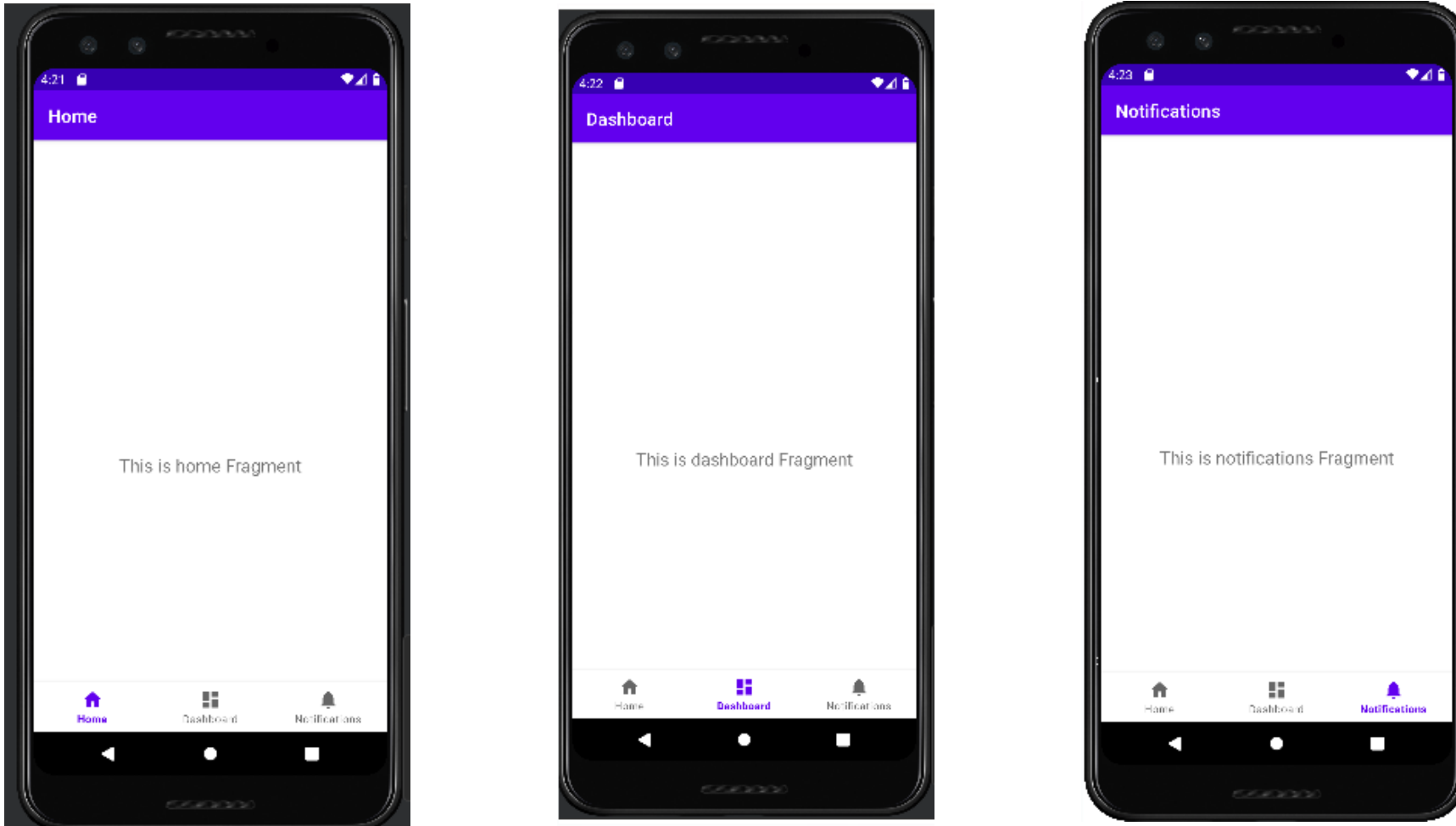
Utilizando la plantilla de
Android Studio



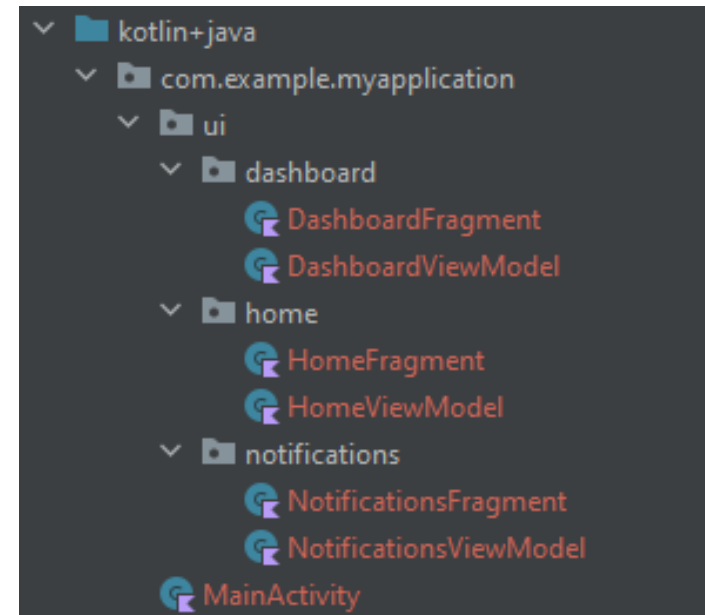
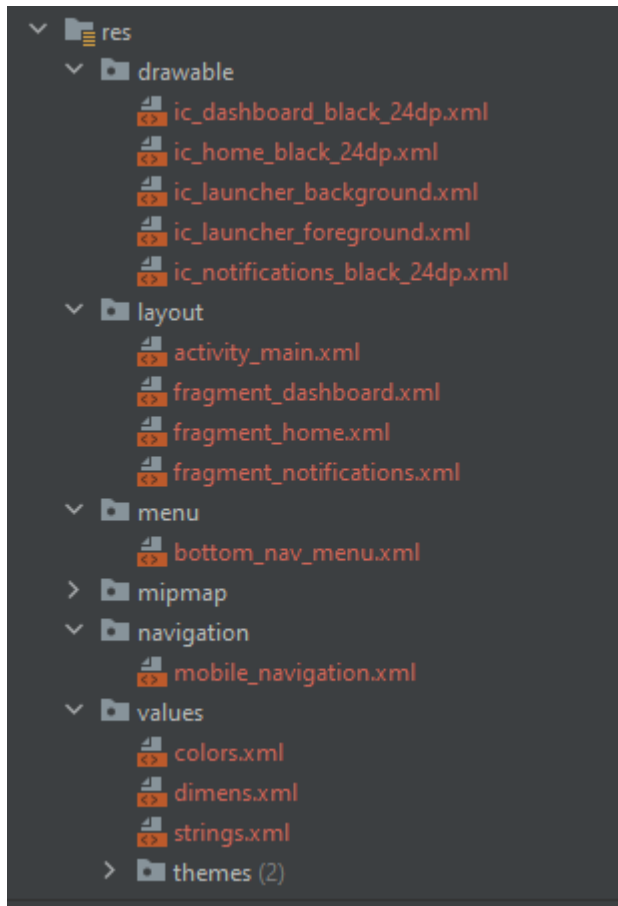
Menú de navegación - Botton Navigation (plantilla)



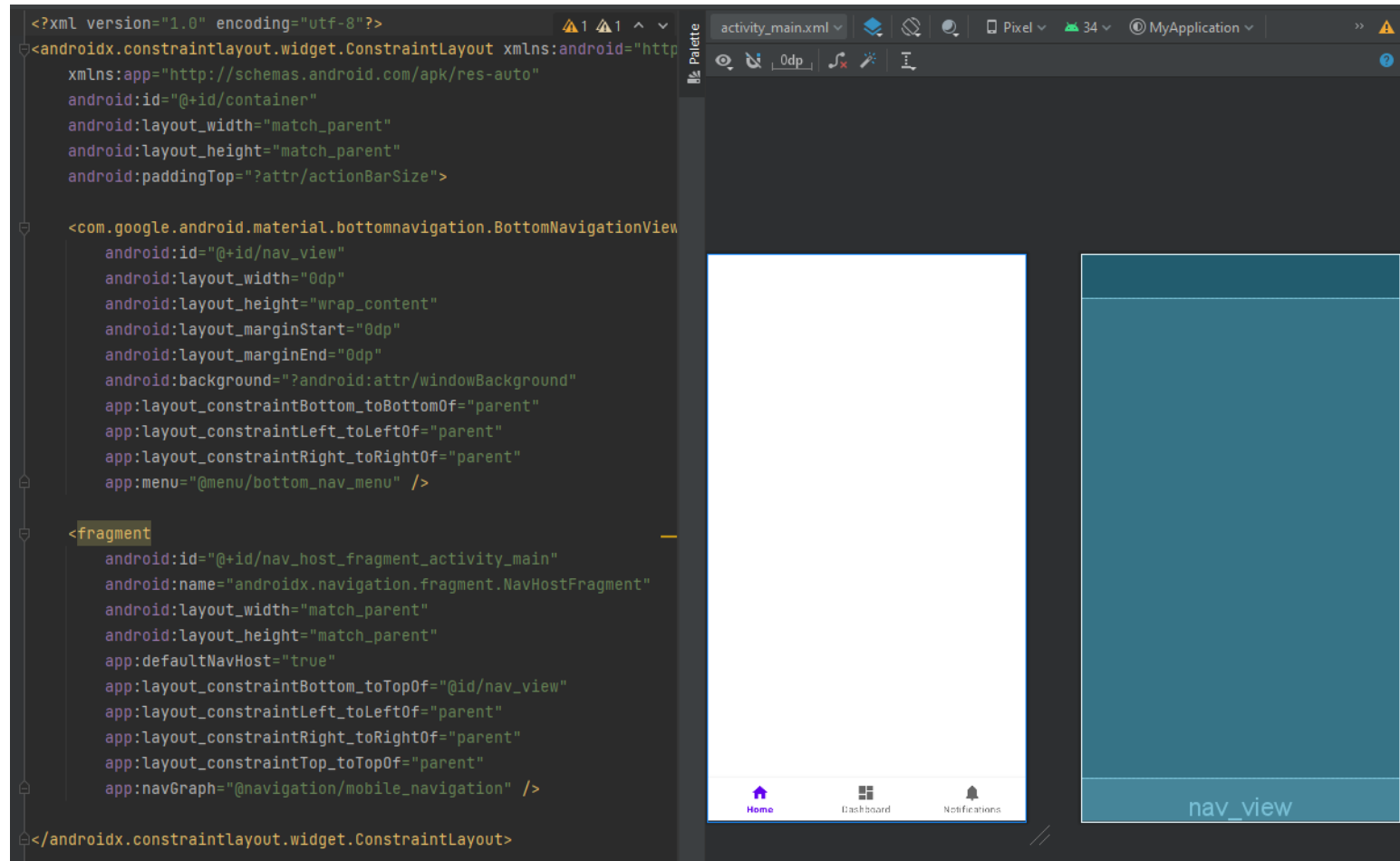
Menú de navegación - Botton Navigation (plantilla)



Menú de navegación - Botton Navigation (plantilla)



Menú de navegación - Botton Navigation (plantilla)



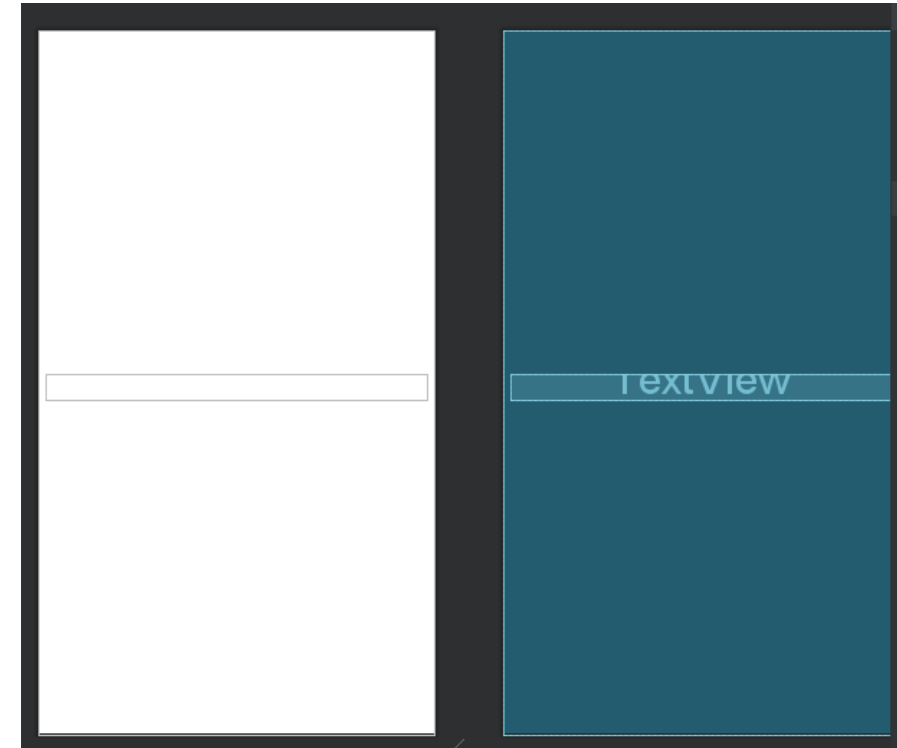
res/layout\

activity_main.xml

Menú de navegación - Botton Navigation (plantilla)

```
<?xml version="1.0" encoding="utf-8" ?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".ui.dashboard.DashboardFragment">

    <TextView
        android:id="@+id/text_dashboard"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginStart="8dp"
        android:layout_marginTop="8dp"
        android:layout_marginEnd="8dp"
        android:textAlignment="center"
        android:textSize="20sp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```



res\layout\

fragment_dashboard/home/notifications.xml

Menú de navegación - Botton Navigation (plantilla)

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/navigation_home"
        android:icon="@drawable/ic_home_black_24dp"
        android:title="Home" />

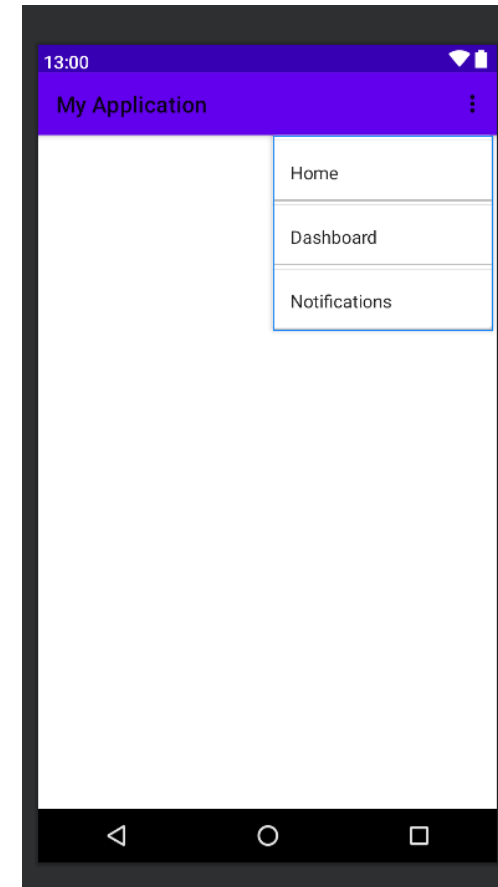
    <item
        android:id="@+id/navigation_dashboard"
        android:icon="@drawable/ic_dashboard_black_24dp"
        android:title="Dashboard" />

    <item
        android:id="@+id/navigation_notifications"
        android:icon="@drawable/ic_notifications_black_24dp"
        android:title="Notifications" />

</menu>
```

res/menu/

bottom_nav_menu.xml



Menú de navegación - Botton Navigation (plantilla)

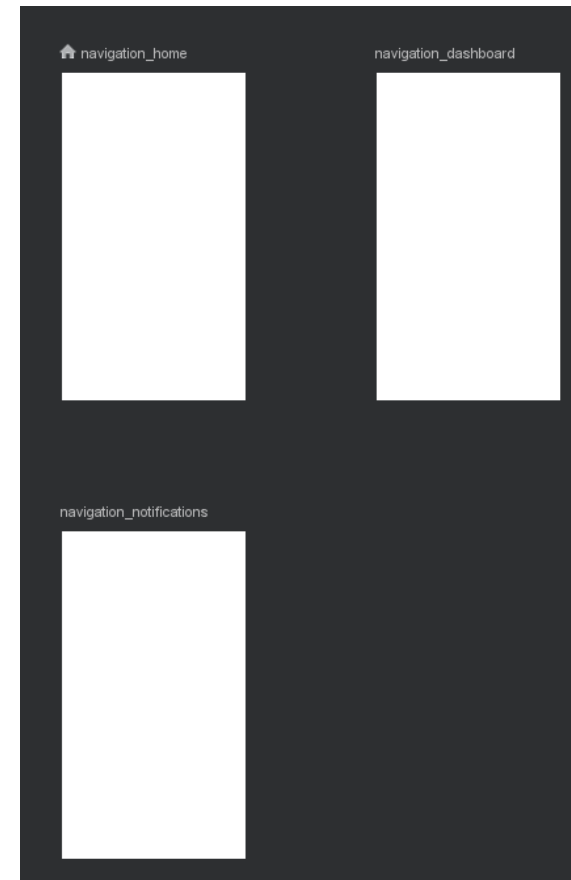
```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/mobile_navigation"
    app:startDestination="@+id/navigation_home">

    <fragment
        android:id="@+id/navigation_home"
        android:name="com.example.myapplication.ui.home.HomeFragment"
        android:label="Home"
        tools:layout="@layout/fragment_home" />

    <fragment
        android:id="@+id/navigation_dashboard"
        android:name="com.example.myapplication.ui.dashboard.DashboardFragment"
        android:label="Dashboard"
        tools:layout="@layout/fragment_dashboard" />

    <fragment
        android:id="@+id/navigation_notifications"
        android:name="com.example.myapplication.ui.notifications.NotificationsFragment"
        android:label="Notifications"
        tools:layout="@layout/fragment_notifications" />

</navigation>
```



res\navigation\

mobile_navigation.xml